MAKERS E DESENVOLVEDORES

Do raciocínio lógico à criação de projetos reais

LIVRO DO ALUNO

Baseado nos repositórios reais de **github.com/Marquezbertin**

Material Didático Gratuito • Aberto • Para uso em sala de aula

Licença MIT

SUMÁRIO

- 1. Pensando como um Programador
- 2. Lógica e Algoritmos
- 3. Python do Zero
- 4. Python na Prática: Projetos
- 5. Desenvolvimento Web: HTML, CSS e JavaScript
- 6. Banco de Dados e SQL
- 7. Git, GitHub e Trabalho em Equipe
- 8. Qualidade de Software e Testes (QA)
- 9. Inteligência Artificial para Iniciantes
- 10. Metodologias Ágeis, Cibersegurança e Ética
- 11. Inglês Técnico para Programadores
- 12. Projeto Social com Impacto na Comunidade
- 13. Glossário
- 14. Referências Bibliográficas

■ Trilha ADOLESCENTES (13–17 anos) — Makers/Devs

Carga horária: 16 aulas × 90 min = 24 horas

Público: "Makers/Devs" (nome do plano pedagógico)

Ambientação: cada aluno com notebook próprio é o ideal; a trilha foi desenhada

para funcionar com 1 PC por 2 alunos (papel → teclado).

SAIBA MAIS: > Fontes: laboratorio-tech-social/trilhas/adolescentes-13-20/,

SAIBA MAIS: > laboratorio-tech-social/site/trilhas/adolescentes/,

SAIBA MAIS: > python-do-zero/, html-css-js-pratica/, banco-de-dados-sql/,

SAIBA MAIS: > git-github-iniciantes/, manual-qa-completo/, ia-para-iniciantes/,

SAIBA MAIS: > ingles-para-programadores/, formacaoScrumMaster/, qa-exercicios-praticos/.

Mapa da trilha (12 módulos, 16 aulas)

| Aula | Módulo | Repo-base | Entregável |

|---|---|---|---|

| 01 | B1 — Pensamento Computacional Avançado | pensamento_computacional | Fluxograma + pseudocódigo de um sistema real |

| 02 | B2 — Lógica e Algoritmos | logica_programacao (80 algoritmos) | 8 algoritmos em pseudocódigo + 2 em Visualg |

EXERCÍCIOS PARA PRATICAR

| 03 | B3 — Python do Zero | python-do-zero (15 módulos/90 exercícios) | 5 scripts rodando |

| 04 | B4 — Python: projects | lista-de-exercicios-python-brasil (112) | 1 mini-jogo + 1 script de automação |

| 05 | B5 — Web: HTML, CSS, JS | html-css-js-pratica (15 módulos) | Site responsivo de 3 páginas |

| 06 | B6 — Banco de Dados e SQL | banco-de-dados-sql (15 módulos) | Schema + 10 consultas do projeto |

| 07 | B7 — Git e GitHub | git-github-iniciantes (15 módulos) | Repositório + 10 commits + README + Pages |

| 08 | B8 — QA: Teste de Software | manual-qa-completo + qa-exercicios-praticos | Plano de teste + 1 bug corrigido |

| 09 | B9 — IA para Iniciantes | ia-para-iniciantes (15 módulos) | 1 chatbot/gerador com prompt avaliado |

MÃO NA MASSA
| 10 | B10 — Metodologias Ágeis (Scrum/Kanban) | formacaoScrumMaster | Sprint funcionando na prática |

| 11 | B11 — Cibersegurança, Ética e Paradigmas | pensamento_computacional/13 e 15 | Política de segurança do projeto + aula aberta |

| 12 | B12 — Inglês Técnico | ingles-para-programadores | README do projeto em inglês |

| 13 | Projeto Final — parte 1 | todos | Escopo + protótipo |

| 14 | Projeto Final — parte 2 | todos | Implementação |

| 15 | Projeto Final — parte 3 | todos | Testes + deploy |

| 16 | Showcase | todos | Apresentação + publicação |

Plano de contingency: se a escola só puder oferecer 8 aulas, use a marca ■ no

mapa acima (aula que é indispensável).

Objetivos do plano pedagógico (Makers/Devs)

1. Dominar uma linguagem de programação (Python) com proficiência intermediária.
2. Criar aplicações web responsivas com HTML, CSS e JavaScript.
3. Escrever e executar testes automatizados entendendo o ciclo de vida do bug.
4. Aplicar metodologias ágeis em projetos colaborativos.
5. Publicar projetos open source que gerem impacto social na comunidade.
6. Compreender implicações éticas de decisões tecnológicas.

Objetivo final do repositório: "Entregar um projeto de impacto social em open source

(..., bot de dados do Esporte São Pedro, ferramenta de QA educativa). Publicado no GitHub Pages."

Regras da sala com adolescentes

| Regra | Por quê |

|---|---|

| Erro é método — o professor pergunta, não entrega | autonomia |

| "Commit ou não existe" | versionamento desde o dia 1 |

| Nada é secreto — README documenta tudo | o README é a avaliação |

| IA é ferramenta, não autor | todo código gerado por IA precisa de revisão humana |

| Nenhum dado real de aluno em repositório público | ver 00-GUIA-DO-PROFESSOR/03-metodologia-e-avaliacao.md §9 |

| Teste que não roda é bug | aprendo QA na própria pele |

Materiais e contas

| Item | Detalhe |

|---|---|

| Python 3.10+ | ou Google Colab — sem instalar nada |

| Navegador | Chrome ou Firefox |

| Editor | VS Code (o repositório laboratorio-tech-social sugere; ou o ID do Colab) |

| Git | instalado no notebook (ou GitHub Desktop) |

| Conta GitHub | https://github.com/signup — obrigatória na aula 07 |

| Conta de IA | a escola define qual usar; ver ia-para-iniciantes/01_ia_generativa/ |

| Scratch | não é mais necessário (a trilha de 8–12 já cobriu) |

> **EXERCÍCIOS PARA PRATICAR**
> Todos os cursos do professor rodam offline: python-do-zero traz os exercícios em .txt com "Resposta Esperada" — pode ser aplicado sem internet, com o aluno testando no Colab depois.

Ambiente de trabalho padrão da turma (o "kit do Maker")

Crie um repositório-base para a turma e distribua:

projeto-da-turma/

■■■ README.md # índice da turma (veja B12 para a versão em inglês)

■■■ docs/

■ ■■■ decisoes.md # ADR simplificado: cada decisão técnica tem data e motivo

■ ■■■setup.md # como rodar o projeto

■■■ src/ # código

■■■ tests/ # testes automatizados

■■■ data/ # dados (CSV/JSON) — sem dados pessoais reais

■■■ notas/ # anotações das aulas

SAIBA MAIS: > Esse esqueleto vem da prática real dos repositórios do autor

SAIBA MAIS: > (qa-automation-framework, central-de-noticias, esporte-sao-pedro).

Avaliação

| Componente | Peso | Onde está o critério |

|---|---|---|

EXERCÍCIOS PARA PRATICAR
| Participação e exercícios | 20% | 04-BANCO-DE-EXERCICIOS/ |

| Commits e histórico Git | 20% | 02-ADOLESCENTES-13-17/07-git-github.md |

| Qualidade de código e testes | 20% | 08-qa-teste-de-software.md |

| Projeto final | 30% | 13-projeto-final.md |

| Mostra / Showcase | 10% | 13-projeto-final.md |

Rubrica detalhada (4 níveis) em 00-GUIA-DO-PROFESSOR/03-metodologia-e-avaliacao.md §4.1.

Slides prontos (do repositório)

| Slide | Arquivo |

|---|---|

| Python | laboratorio-tech-social/materiais/slides/aula-teens-01-python.html |

| Web | laboratorio-tech-social/materiais/slides/aula-teens-02-web.html |

| QA | laboratorio-tech-social/materiais/slides/aula-teens-03-qa.html |

| Dados | laboratorio-tech-social/materiais/slides/aula-teens-04-dados.html |

| IoT | laboratorio-tech-social/materiais/slides/aula-teens-05-iot.html |

| Projeto | laboratorio-tech-social/materiais/slides/aula-teens-06-projeto.html |

| História | laboratorio-tech-social/site/slides/aula-teens-07-historia-computacao.html |

Trilha web da turma (13–20): laboratorio-tech-social/site/trilhas/adolescentes/

→ 01-python.html, 02-web.html, 03-qa.html, 04-dados.html, 05-iot.html,

06-projeto.html, 07-historia-computacao.html, 08-informatica.html.

Módulos em markdown do repositório:

trilhas/adolescentes-13-20/modulo-03-qa-testes.md,

modulo-04-ciencia-dados.md, modulo-05-iot-nuvem.md, modulo-06-projeto-social.md.

Onde a turma Treina por conta (games)

QA_RUN/ — 71 jogos educativos de QA, HTML5, jogáveis no navegador.

Os mais úteis para esta trilha:

| Jogo | Caminho | Conceito |

|---|---|---|

| Quiz Duel | QA_RUN/js/games/quizduel.js | Conceitos de QA |

| Caça-Bugs | laboratorio-tech-social/site/exercicios/caca-bugs.html | Encontrar bugs |

| Plano de Testes | laboratorio-tech-social/site/exercicios/plano-testes.html | Escrever plano de teste |

| Simulador de API | laboratorio-tech-social/site/exercicios/simulador-api.html | API e status HTTP |

| Validador de Regex | laboratorio-tech-social/site/exercicios/validador-regex.html | Expressões regulares |

EXERCÍCIOS PARA PRATICAR
| Desafios Python | laboratorio-tech-social/site/exercicios/desafios-python.html | Fixação de sintaxe |

| Code Builder | laboratorio-tech-social/site/exercicios/code-builder.html | Montar lógica em blocos |

| Simulador de Robótica | laboratorio-tech-social/site/exercicios/simulador-robotica.html | Sensores e automação |

| Despertar da Qualidade | QA_RUN/js/games/quizduel.js + questionario_despertardaqualidade/ | Quiz de 10 questões V/F |

■ Próxima trilha

03-JOVENS-E-ADULTOS-18+

B1 — Pensamento Computacional Avançado (13–17 anos)

Duração: 90 min · Aula 1 de 16

■ Objetivo

Ao final da aula, o aluno será capaz de decompor um sistema real (ex.: o sistema de matrícula da escola) em 4 camadas — dados, regras, interface, infraestrutura — e escrever o pseudocódigo de 1 regra de negócio com condicional + repetição.

Objetivo secundário: Reconhecer os 4 pilares do pensamento computacional em código que já existe (e apontar onde falta um deles).

■ Fontes no repositório

| Arquivo | O que traz |

|---|---|

| pensamento_computacional/01_pensamento_computacional/exercicio_04_quatro_pilares.txt | Os 4 pilares — definição oficial do repo |

EXERCÍCIOS PARA PRATICAR

| pensamento_computacional/02_decomposicao/ (6 exercícios) | Decomposição aplicada a casos reais |

| pensamento_computacional/03_reconhecimento_padroes/ (6) | Padroes: quando repetir codigo, quando abstrair |

| pensamento_computacional/04_abstracao/ (6) | Abstração: o que esconder e por quê |

| pensamento_computacional/05_pensamento_algoritmico/ (6) | Algoritmo, decisões, repetições, otimização |

| pensamento_computacional/06_fluxogramas/ (6) | Fluxogramas de sistemas com várias decisões |

| pensamento_computacional/07_pseudocodigo/ (6) | Pseudocódigo completo, do "hello world" a programa com lista |

| pensamento_computacional/08_logica_proposicional/ (6) | Lógica proposicional: E, OU, combinação, paradoxos |

| laboratorio-tech-social/site/exercicios/pensamento-computacional.html | Jogo interativo de pensamento computacional |

| laboratorio-tech-social/site/exercicios/code-builder.html | Montar lógica em blocos |

| laboratorio-tech-social/site/exercicios/criador-projeto.html | Estruturar um projeto antes de codar |

■ O framework que a aula ensina

O pensamento_computacional organiza tudo em 4 pilares. Na faixa 8–12 você usou os

pilares como jogo. Agora eles viram ferramenta de engenharia:

| Pilar | No código | Pergunta de revisão |

|---|---|---|

| Decomposição | Funções, classes, módulos, serviços | "Isso está grande demais? Onde corto?" |

| Reconhecimento de padrões | Funções reutilizáveis, for, listas, ORM, APIs | "Esse código já existe? Onde?" |

| Abstração | Interfaces, classes abstratas, dict em vez de 50 variáveis | "O que o chamador precisa saber?" |

| Algoritmos | O for, o if, a função, a query SQL | "Os passos estão certainos e na ordem?" |

A tabela de decomposição em 4 camadas (adaptada de 12_modelagem_er do repo SQL e do modulo-06-projeto-social.md):

| Camada | O que é | Exemplo (escola) |

|---|---|---|

| Dados | O que é guardado | Aluno, Turma,Matrícula,Nota,Professor |

| Regras | O que é permitido/calculado | "Nota < 5 reprova"; "Turma <= 40 alunos" |

| Interface | Como o humano usa | Formulário de matrícula, relatório |

| Infraestrutura | Onde roda | Servidor, banco, CI, deploy |

■■ Roteiro (90 min)

Fase 1 — Apresentação (10 min)

Gancho: "Vocês já usaram o sistema de matrícula da escola. Ele tem tela, tem banco de dados, tem regra que reprova aluno. Se alguém apagar essa regra sem querer, ninguém nota até o boletim sair errado. Hoje a gente vai abrir esse sistema e ver as 4 camadas."

Quiz de recuperação (Aula 0 / do 8–12): "O que é decomposição?"

Demonstração ao vivo (5 min): mostre o banco-de-dados-sql/14_projeto_sistema_escolar/ e pergunte: "Onde está a regra que reprova? Onde está o dado? Onde está a tela?"

Fase 2 — Exploração (20 min)

Parte 1 — Os 4 pilares em código real (10 min).

Abra html-css-js-pratica/14_projeto_lista_tarefas/ e identifique com a turma:

- - Decomposição: funções separadas (adicionarTarefa, renderizarLista, salvar)
- - Padrão: o mesmo "adicionar item na lista" aparece em vários lugares → virou função
- - Abstração: quem chama salvar() não sabe se é localStorage ou API

- - Algoritmo: o for que percorre a lista e cria os

Parte 2 — Lógica proposicional (10 min).

Base: pensamento_computacional/08_logica_proposicional/.

EXERCÍCIOS PARA PRATICAR
Exercício ao vivo: "O aluno pode ser reprovado se: nota < 5 E não fez arecovery

OU faltou 60% das aulas. Escreva a condição."

python

nota < 5 and not fez_recovery or faltas > 0.6 aulas_totais

SAIBA MAIS: > A pegadinha clássica: sem parênteses, a ordem das operações muda tudo.

SAIBA MAIS: > A and B or C = (A and B) or C. É a mesma armadilha do fluxograma da A2.

SAIBA MAIS: > Peça: "desenhe o fluxograma dessa condição e depois escreva o código". Se o código e o

SAIBA MAIS: > fluxograma não baterem, achamos um bug.

Fase 3 — Mão na massa (35 min)

MÃO NA MASSA
Atividade A — Decomposição do sistema da escola (20 min).

Em duplas, preencher o quadro (1 folha A3 por dupla):

SISTEMA ESCOLAR — Decomposição em 4 camadas

DADOS (o que é guardado):

Entidade | Atributos (3+) | Tipo

----------|----------|----------

Aluno | |

Turma | |

----------+----------+----------

REGRAS (o que é permitido/calculado):

INTERFACE (o que a pessoa usa):

Tela 1:

Tela 2:

INFRAESTRUTURA (onde roda):

App:

Banco:

Deploy:

DADO QUE NÃO DEVERIA EXISTIR:

(ex.: senha em texto puro, nota de aluno junto com documento)

SAIBA MAIS: > O último campo é o que diferencia um adolescente de um adulto. Descomente: "O sistema

SAIBA MAIS: > real guarda isso. E nós vamos fazer o nosso sem guardar."

MÃO NA MASSA
Atividade B — Pseudocódigo de 1 regra (15 min).

Cada dupla escreve o pseudocódigo de 1 regra do quadro acima, no formato do repo

(07_pseudocodigo/):

ALGORITMO: ValidarMatricula

ENTRADA: aluno, turma

SAIDA: pode_matricular (VERDADEIRO ou FALSO)

INICIO

SE turma.vagas <= 0 ENTAO

RETORNAR FALSO

FIM SE

SE aluno.ja_matriculado EM turma ENTAO

RETORNAR FALSO

FIM SE

SE turma.tem_turno_incompativel COM aluno.turno ENTAO

RETORNAR FALSO

FIM SE

turma.vagas <- turma.vagas - 1

RETORNAR VERDADEIRO

Fase 4 — Projeto (20 min)

EXERCÍCIOS PARA PRATICAR
Desafio: "Diagnóstico de um repositório"

Cada dupla escolhe 1 repositório real do autor e faz a decomposição em 4 camadas:

Opções (todas existentes e auditadas):

- - central-de-noticias — pipeline RSS → resumo → TTS → PWA
- - esporte-sao-pedro — Supabase + RLS + Edge Functions + IA + SEO
- - meu-treino — PWA com Wake Lock, GPS, Web Share, localStorage
- - anyms — gestão com IndexedDB, 17 tabelas, custo médio ponderado
- - hubias-estudos — SQLite (WASM) no navegador + flashcards + Pomodoro + sync

MÃO NA MASSA
Entrega da atividade:

- - [ ] 1 folha com as 4 camadas preenchidas
- - [ ] 1 regra real escrita em pseudocódigo
- - [ ] 1 problema encontrado (é o gancho para a aula de QA)

Fase 5 — Mostra (5 min)

2 duplas apresentam. A turma responde a pergunta: "Qual camada o bug mais provavel esta?"

MÃO NA MASSA
(o bug encontrado na atividade vira matéria-prima para a aula 08 de QA).

■ Avaliação

| Instrumento | Critério |

|---|---|

| Quadro de decomposição | 4 camadas preenchidas + 1 dado indevido identificado |

| Pseudocódigo | tem ENTRADA, SAÍDA, 1 condicional, 1 retorno |

| Diagnóstico de repositório | 4 camadas + 1 problema real encontrado |

| Participação | 1 pergunta ao vivo por aluno |

| Pergunta de saída | "Qual a diferença entre dado e regra?" |

Rubrica (4 níveis) — critério "Decomposição":

Semente = nomeia as 4 camadas / Broto = preenche 3 / Árvore = preenche 4 + pseudocódigo /

Floresta = preenche 4 + pseudocódigo + identifica dado indevido + problema real.

■ Exercícios extras (do repositório)

| Arquivo | Sobre |

|---|---|

| 02_decomposicao/exercicio_03_criar_site.txt | Decomposição de um site |

| 02_decomposicao/exercicio_04_organizar_evento.txt | Decomposição de um evento (bom para a escola) |

| 02_decomposicao/exercicio_06_cinco_porques.txt | Causa raiz — base do mindset de QA |

| 03_reconhecimento_padroes/exercicio_04_tendencias.txt | Identificar o que se repete num código |

| 04_abstracao/exercicio_03_generalizar.txt | O que generalizar |

| 04_abstracao/exercicio_05_simplificar.txt | Simplificar sem perder comportamento |

| 05_pensamento_algoritmico/exercicio_06_otimizacao.txt | Complexidade: o que muda quando o dado dobra |

| 08_logica_proposicional/exercicio_06_paradoxos.txt | Paradoxo do barbeiro (puro pensar) |

EXERCÍCIOS PARA PRATICAR
| logica_programacao/15_exercicios_desafio/ | Desafios finais de lógica |

■ Para casa

Escolher um app que você usa todo dia e fazer a decomposição em 4 camadas.

Trazer impresso ou no celular.

■ Próximo módulo

B2 — Lógica e Algoritmos

B2 — Lógica e Algoritmos (13–17 anos)

Duração: 90 min · Aula 2 de 16

■ Objetivo

Ao final da aula, o aluno será capaz de:

1. Escrever 8 algoritmos em pseudocódigo cobrindo variáveis, condicionais, laços, vetores, matrizes, strings, procedimentos, funções, recursão, ordenação e busca.
2. Implementar 2 deles em Visualg (os que rodam no executável da escola).
3. Explicar a diferença entre busca linear e busca binária com medição de tempo.

Objetivo secundário: Reconhecer recursão e saber escrever 1 função recursiva simples.

■ Fontes no repositório

logica_programacao/ — 15 módulos e 80 algoritmos (a base mais completa do material):

| Módulo | Conteúdo | Nº de algoritmos |

|---|---|---|

| 01_variaveis_e_tipos | Variáveis, tipos (inteiro, real, caracter, lógico) | — |

| 02_entrada_saida | escreva, leia, entrada/saída | — |

| 03_operadores | Aritméticos, lógicos, relacionais | — |

| 04_condicionais | se, se...senao, escolha | — |

| 05_lacos_repeticao | enquanto, para, repita | — |

| 06_vetores | Vetores de 1, 2 e 3 dimensões | — |

| 07_matrizes | Matrizes (matriz x vetor) | — |

| 08_strings | Manipulação de texto | — |

| 09_procedimentos | Procedimentos (sem retorno) | — |

| 10_funcoes | Funções (com retorno) | — |

| 11_recursao | Recursão | — |

| 12_ordenacao | Bubble sort, insertion sort, seleção | — |

| 13_busca | Busca linear, busca binária | — |

| 14_introducao_a_dados | Vetores e matrizes em profundidade | — |

EXERCÍCIOS PARA PRATICAR
| 15_exercicios_desafio | Desafios finais | — |

Complementos:

- - laboratorio-tech-social/site/exercicios/code-builder.html — montador de lógica em blocos
- - laboratorio-tech-social/site/exercicios/validador-regex.html — expressões regulares
- - pensamento_computacional/06_fluxogramas/ e 07_pseudocodigo/

■ Conceitos-chave

| Conceito | Por que importa |

|---|---|

| Vetor | Guardar vários valores do mesmo tipo. É a base de lista, array e tabela |

| Matriz | Vetor de vetores. É a base de planilha, imagem e tabela de banco |

| Procedimento vs Função | Procedimento executa; função devolve valor. Distinção que a turma esquece sempre |

| Recursão | Um caso chama a si mesmo com um caso base. Base de árvore, grafo e recursão de pastas |

| Complexidade | O(n) vs O(n log n). Aula que separa quem só decora de quem entende |

Analogias:

- - Vetor = prateleira com gavetas numeradas (acesso direto: gaveta 5)
- - Busca linear = procurar um livro varrendo a prateleira da esquerda para a direita
- - Busca binaria = a adivinhacao do numero - metade sempre

■■ Roteiro (90 min)

Fase 1 — Presentação (10 min)

Gancho: "Seu buscador favorite resolve em 0,2 s. Eu vou te mostrar por que

buscar 1.000.000 de registros 'na mão' leva 1.000.000 de passos."

Demonstração ao vivo (5 min): escreva no quadro a busca linear e rode mentalmente:

ENCONTRE(x em v[0..n-1]):

para i de 0 até n-1:

se v[i] == x: retorne i

retorne -1

Pergunta: "Quantos passos, no pior caso, para achar o último item?" → n.

"E se eu achar que está na metade?" → log■n = ~20 passos para 1.000.000.

"Essas são as 2 respostas que a aula vai construir."

Fase 2 — Exploração (20 min)

Parte 1 — Vetores e matrizes (8 min).

Mostre que a planilha da trilha 8–12 já é uma matriz, e que a lista de tarefas do

html-css-js-pratica/14_projeto_lista_tarefas/ já é um vetor.

Peça: "Dê 3 exemplos de vetor que vocês usam todo dia" (lista de amigos, notas, vídeos).

Parte 2 — Procedimento vs Função (6 min).

PROCEDIMENTO MostrarMensagem(texto): # executa, não devolve nada

escreva(texto)

FUNCAO Somar(a, b): # devolve valor

retorne a + b

Pergunta: "Qual dos dois eu uso quando o resultado entra numa fórmula?"

Parte 3 — Recursão (6 min).

Mostre a soma dos números de 1 a n:

FUNCAO Soma(n):

se n == 0: retorne 0 # CASO BASE

retorne n + Soma(n - 1) # CHAMADA RECURSIVA

Rastreie Soma(4) na lousa: 4+3+2+1+0 = 10.

A analogia: boneca russa — cada uma contém uma menor até a menor não conter mais nada.

SAIBA MAIS: > Armadilha clássica: sem caso base → pilha estoura (RecursionError).

SAIBA MAIS: > Teste no Colab: escreva def f(n): return f(n-1) e chame f(1). Deixe quebrar na frente da turma.

Fase 3 — Mão na massa (35 min)

MÃO NA MASSA
Atividade A — 8 algoritmos em pseudocódigo (25 min).

Cada aluno pega a folha "Algoritmos" (modelo abaixo) e escreve 8 algoritmos,

1 por módulo do repo. Escolha 8 entre:

| # | Módulo de origem | Algoritmo a escrever |

|---|---|---|

| 1 | 01_variaveis_e_tipos | Trocar 2 valores de 2 variáveis (sem variável temporária) |

| 2 | 02_entrada_saida | Ler nome e 3 notas, mostrar a média |

| 3 | 03_operadores | Verificar se um número é par E múltiplo de 3 |

| 4 | 04_condicionais | Classificar nota (A/B/C/D/E) |

| 5 | 05_lacos_repeticao | Somar todos os números pares de 1 a 100 |

| 6 | 06_vetores | Preencher um vetor de 10 números e mostrar o maior e o menor |

| 7 | 07_matrizes | Preencher uma matriz 3x3 e mostrar a diagonal principal |

| 8 | 08_strings | Contar quantas vogais tem uma frase |

| 9 | 09_procedimentos | Procedimento MostarMenu() que exibe 3 opções |

| 10 | 10_funcoes | Função EhPrimo(n) que retorna V ou F |

| 11 | 11_recursao | Fatorial(n) recursivo |

| 12 | 12_ordenacao | Bubble sort de um vetor de 5 números |

| 13 | 13_busca | Busca binária num vetor ordenado |

| 14 | 14_introducao_a_dados | Preencher matriz de notas de 3 alunos × 4 disciplinas e calcular a média de cada aluno |

SAIBA MAIS: > Critério: pseudocódigo do padrão logica_programacao (ENTRADA / SAIDA / INICIO / FIM),

SAIBA MAIS: > com indentação e 1 algoritmo por bloco.

MÃO NA MASSA
Atividade B — 2 algoritmos em Visualg (10 min).

Se a escola tiver o Visualg instalado (o repo logica_programacao foi feito para ele):

implemente o #2 (média) e o #5 (pares de 1 a 100).

Se não tiver: implemente no Scratch ou em Python (Colab).

Fase 4 — Projeto (20 min)

EXERCÍCIOS PARA PRATICAR
Desafio: "O caçador de números"

Implemente (em pseudocódigo + implementação) um programa que:

1. Leia 20 números para um vetor.
2. Peça um valor de busca.
3. Use busca linear e busca binária (ordene antes).
4. Diga se encontrou, em qual posição (1-based) e quantas comparações cada método fez.

EXERCÍCIOS PARA PRATICAR
Desafio bônus: rode com 1.000, 10.000 e 100.000 números e anote as comparações.

A turma vai ver a curva n × log n desenhada na lousa.

Fase 5 — Mostra (5 min)

3 alunos apresentam um dos 8 algoritmos. Turma identifica: qual módulo do repo ele veio de.

■ Avaliação

| Instrumento | Critério |

|---|---|

| 8 algoritmos | 8 de 8 no formato correto, com ENTRADA/SAIDA quando aplicável |

| 2 implementações | Rodam sem erro |

| Caçador de números | Mostra posição + contagem de comparações nos 2 métodos |

| Participação | 1 algoritmo explicado por aluno (oral) |

| Pergunta de escrita | "Complexidade da busca linear vs binária" |

Pergunta de escrita (vale 1 ponto):

> **SAIBA MAIS:** > "Você precisa procurar 1 nome em uma lista de 10.000 contatos que não está ordenada.
>
> **SAIBA MAIS:** > Busca linear ou binária? Justifique."

Resposta esperada: binária não serve (exige ordenação). Ou: se ordenar primeiro, ordenar custa O(n log n) e não compensa para 1 busca. É a resposta que mostra entendimento.

■ Exercícios extras (do repositório)

| Arquivo | Sobre |

|---|---|

> **EXERCÍCIOS PARA PRATICAR**
> | logica_programacao/15_exercicios_desafio/ | Desafios finais (use como banco de exercícios extras) |
>
> **EXERCÍCIOS PARA PRATICAR**
> | logica_programacao/11_recursao/ | Exercícios de recursão (fibonacci, hanoi, torre) |

| logica_programacao/12_ordenacao/ | Comparação entre os 3 algoritmos de ordenação |

| logica_programacao/13_busca/ | Busca sequencial × binária |

> **EXERCÍCIOS PARA PRATICAR**
> | laboratorio-tech-social/site/exercicios/desafios-python.html | Versão em Python dos desafios |

| pensamento_computacional/05_pensamento_algoritmico/exercicio_06_otimizacao.txt | Otimização |

■ Para casa

Escolha 2 dos 8 algoritmos e implemente em Python no Google Colab.

Traga o link do notebook (ou o código) na próxima aula.

■ Próximo módulo

B3 — Python do Zero

B3 — Python do Zero (13–17 anos)

Duração: 90 min · Aula 3 de 16

■ Objetivo

Ao final da aula, o aluno será capaz de:

1. Escrever e executar um programa Python com entrada, saída, variável, condicional, laço, lista e função — sem colar Stack Overflow.
2. Ler e explicar um trace (execução passo a passo) de um programa de 15 linhas.

EXERCÍCIOS PARA PRATICAR
3. Ler a "Resposta Esperada" de um exercício do repo e autoavaliar sua solução.

Objetivo secundário: Ler mensagens de erro do Python em inglês e sacar o que elas dizem.

■ Fontes no repositório

EXERCÍCIOS PARA PRATICAR
python-do-zero/ — 15 módulos × 6 exercícios = 90 exercícios, todos com "Resposta Esperada":

EXERCÍCIOS PARA PRATICAR

| Módulo | Conteúdo | Exercícios (nomes de arquivo) |

|---|---|---|

| 01_introducao | O que é Python, primeiro programa, IDE | exercicio_01_.txt … |

| 02_entrada_saida | input(), print(), tipos de saída | |

| 03_operadores | Aritméticos, comparação, lógicos, +=, , f-string | |

| 04_condicionais | if, elif, else, operadores ternários | |

| 05_repeticoes | for, range, while, break, continue | |

| 06_listas | Criar, acessar, percorrer, append, remove, fatiamento | |

| 07_tuplas_dicionarios | Tupla, dicionário, .get(), percorrer chave/valor | |

| 08_funcoes | def, parâmetros, retorno, escopo, padrão | |

| 09_strings | Fatiamento, métodos (upper, split, replace, join) | |

| 10_arquivos | Abrir, ler, escrever, with, CSV | |

| 11_tratamento_erros | try/except/finally, exceções | |

| 12_modulos | import, from, módulos da biblioteca padrão | |

| 13_programacao_orientada_objetos | Classes, __init__, métodos, herança | |

| 14_projetos_praticos | 6 projetos completos | |

| 15_api_e_json | Consumo de API, JSON, requests | |

Complementos:

- - laboratorio-tech-social/site/trilhas/adolescentes/01-python.html — trilha web
- - laboratorio-tech-social/materiais/slides/aula-teens-01-python.html — slide

EXERCÍCIOS PARA PRATICAR

- laboratorio-tech-social/site/exercicios/desafios-python.html — exercícios no navegador

- - laboratorio-tech-social/site/exercicios/flashcards.html — fixação por cartões

EXERCÍCIOS PARA PRATICAR

- lista-de-exercicios-python-brasil/ — 112 exercícios (6 seções), com doctests

- - Calculadora/, jogo_da_velha/, o_acolito/, anyms/ — projetos reais do autor

■ Conceitos-chave

| Conceito | Frase que fixa |

|---|---|

| Variável | "Uma caixa com nome dentro. O nome é o rotulo, o conteúdo muda." |

| Tipo | "Python descobre o tipo sozinho. Você não declara." |

| Indentação | "Em Python, espaço no começo da linha é sintaxe. Erro de espaço = programa errado." |

| input() sempre retorna texto | "Sempre. Sempre. Sempre. Precisa converter." |

| f-string | f"Ola {nome}" — o jeito moderno de juntar texto |

| Lista vs Dicionário | "Lista = gavetas numeradas. Dicionário = gavetas com etiqueta." |

| Função | "Uma caixa que recebe, faz, devolve. Sem devolução, é procedimento." |

| with | "Abre, usa e fecha sozinho. Mesmo se der erro." |

■■ Roteiro (90 min)

Fase 1 — Apresentação (10 min)

Gancho: "O programa de vocês vai ler o nome, perguntar 3 coisas e imprimir um resultado.

Parece bobo. Mas é exatamente a estrutura de TODO programa do mundo."

Demonstração ao vivo (5 min): no Colab, digite com erros de propósito:

python

print("Ola" nome)

A turma vê o SyntaxError. Pergunte: "O que ele está reclamando?"

Explique: em Python, (precisa fechar. O erro de digitação é o erro de sintaxe.

Outra demo (3 min):

python

idade = input("Idade: ")

print(idade + 1)

TypeError: can only concatenate str (not "int") to str.

Diga: "Erro em Python não é um problema. É o Python te mostrando onde está o problema.

Traduzir a mensagem é 80% do trabalho de programmer."

Dica de ouro (o repo ingles-para-programadores é exatamente sobre isso):

> **SAIBA MAIS:** > "A documentação e os erros estão em inglês. Aprender a ler erro em inglês é aprender a programar."

Fase 2 — Exploração (20 min)

Parte 1 — A anatomia de um programa (8 min).

Escreva na lousa e peça para a turma identificar as 7 partes:

python

1. comentario

nome = input("Como voce se chama? ") # 2. entrada

notas = [] # 3. lista

for i in range(3): # 4. laco

notas.append(float(input(f"Nota {i+1}: "))) # 5. entrada dentro do laco

media = sum(notas) / len(notas) # 6. calculo

if media >= 7: # 7. condicional

situacao = "aprovado"

else:

situacao = "reprovado"

print(f"{nome}: media {media:.2f} - {situacao}") # 8. saida com f-string

Perguntas: "Qual linha precisa de indentação?" / "Onde está o input?" /

"O que o :2f faz?" (2 casas decimais)

Parte 2 — Lista vs Dicionário (7 min).

python

LISTA: indice numerico

alunos = ["Ana", "Bruno", "Carla"]

print(alunos[0]) # Ana

DICIONARIO: chave -> valor

notas = {"Ana": 8.5, "Bruno": 6.0, "Carla": 9.5}

print(notas["Ana"]) # 8.5

print(notas.get("Dani", 0)) # 0 <- .get() nao quebra se nao existe

Pergunta: "Qual você usaria para uma lista depresence? E para o cadastro de cada aluno?"

Parte 3 — Lendo erro (5 min).

Mostre 4 erros típicos e o que significam:

| Erro | Significado | O que fazer |

|---|---|---|

| SyntaxError | Falta : ou parêntese, ou indentação errada | Releia a linha do traceback |

| NameError: name 'x' is not defined | Você usou x antes de criar | Verifique a ordem das linhas |

| TypeError | Tipo errado (texto com número) | int(), float(), str() |

| IndentationError | Espaço faltando ou sobrando | Regra: 4 espaços |

SAIBA MAIS: > Método do repo ingles-para-programadores/08_comandos_ingles/: leia a mensagem como

SAIBA MAIS: > uma frase: "cannot convert str to float" = "não consegui converter texto em número".

Fase 3 — Mão na massa (35 min)

MÃO NA MASSA
Atividade A — Do módulo 01 ao 05 (20 min).

EXERCÍCIOS PARA PRATICAR
No Colab (ou Python local), faça 5 exercícios de python-do-zero, 1 de cada módulo:

EXERCÍCIOS PARA PRATICAR
| Módulo | Exercício sugerido | Como achar |

|---|---|---|

| 01_introducao | Primeiro programa | exercicio_01_.txt do módulo |

| 02_entrada_saida | Formatar saída com f-string | exercicio_02_.txt |

| 03_operadores | Calculadora de média com 3 operadores | exercicio_03_.txt |

| 04_condicionais | Classificar por faixa de idade | exercicio_04_.txt |

| 05_repeticoes | Somar 1 a N com for e depois com while | exercicio_05_.txt |

EXERCÍCIOS PARA PRATICAR
Regra da turma: para cada exercício, o aluno cola a "Resposta Esperada" do repo no

começo do arquivo e compara com a saída real.

MÃO NA MASSA
Atividade B — Funções (15 min).

Do módulo 08_funcoes, implemente:

python

def media(notas):

return sum(notas) / len(notas)

def classificacao(m):

if m >= 9: return "A"

elif m >= 7: return "B"

elif m >= 5: return "C"

else: return "D"

alunos = {"Ana": [8,9,10], "Bruno": [5,6,4], "Carla": [10,10,9]}

for nome, notas in alunos.items():

print(f"{nome}: {media(notas):.1f} - {classificacao(media(notas))}")

Pergunta: "Por que classificacao recebe um número e não a lista?" (→ função pura, testável)

Fase 4 — Projeto (20 min)

EXERCÍCIOS PARA PRATICAR
Desafio: "Sua ficha de estudante"

Escreva um programa que:

1. Peça nome, idade, 3 notas e 1 cidade.
2. Valida cada entrada (idade entre 0 e 120, nota entre 0 e 10) usando uma função que devolve True/False.
3. Calcula a média com f-string formatado.
4. Salve os dados em uma lista de dicionários.
5. No final, mostre todos os cadastros em tabela (com f"{:<20}" para alinhar).

Critério de pronto:

- - [ ] Usa função para a validação (não repete o if 4 vezes)
- - [ ] Usa f-string em toda saída
- - [ ] Guarda em lista de dicionários
- - [ ] Nenhum erro ao rodar com entradas inválidas
- - [ ] Entrada inválida não quebra o programa (repete a pergunta)

Fase 5 — Mostra (5 min)

2 alunos rodam seu programa ao vivo, um deles digita uma entrada inválida de propósito para mostrar que o programa pede a entrada de novo. A turma ve a robustez.

■ Avaliação

| Instrumento | Critério |

|---|---|

EXERCÍCIOS PARA PRATICAR
| 5 exercícios do repo | "Resposta Esperada" coincide com a saída real |

| Funções | 2 funções implementadas e usadas |

| Ficha de estudante | 5 critérios do "pronto" |

| Leitura de erro | 3 erros explained corretamente (quiz rápido) |

| Pergunta de saída | "O que input() devolve e por que idade + 1 quebra?" |

Quiz rápido (5 min, oral):

1. input() devolve que tipo? → str (sempre)
2. Como descubro o tipo de uma variável? → type(x)
3. O que faz //? → Divisão inteira
4. O que é indentação? → Bloco de código; em Python é sintaxe (4 espaços)
5. notas[0] se notas é vazio? → IndexError

■ Exercícios extras (do repositório)

| Arquivo | Sobre |

|---|---|

| python-do-zero/06_listas/ | Fatiamento, comprehensions |

| python-do-zero/07_tuplas_dicionarios/ | Dicionários aninhados |

| python-do-zero/09_strings/ | Limpeza de texto (útil para o projeto de dados) |

| python-do-zero/10_arquivos/ | Ler/escrever CSV (base do projeto final) |

| python-do-zero/11_tratamento_erros/ | try/except (obrigatório no projeto final) |

| python-do-zero/14_projetos_praticos/ | 6 projetos prontos para adaptar |

| python-do-zero/15_api_e_json/ | Consumo de API (base do bot de notícias) |

EXERCÍCIOS PARA PRATICAR
| lista-de-exercicios-python-brasil/ | 112 exercícios com doctest — banco de exercícios |

EXERCÍCIOS PARA PRATICAR
| laboratorio-tech-social/site/exercicios/desafios-python.html | Exercícios rodando no navegador |

■ Para casa

Refazer a "Ficha de Estudante" em VS Code (não no Colab) e subir para o GitHub

(depois da aula 07). Traga a pasta.

■ Próximo módulo

B4 — Python: projetos práticos

B4 — Python: projetos práticos (13–17 anos)

Duração: 90 min · Aula 4 de 16

■ Objetivo

Ao final da aula, o aluno será capaz de:

EXERCÍCIOS PARA PRATICAR
1. Escolher, adaptar e executar 3 exercícios do banco lista-de-exercicios-python-brasil (112 exercícios, com doctest).

EXERCÍCIOS PARA PRATICAR
2. Transformar um exercício em uma solução completa: entrada validada + função + saída formatada + try/except.

EXERCÍCIOS PARA PRATICAR
3. Publicar 1 Pull Request em um repositório open source resolvendo um exercício vazio.

Objetivo secundário: Ler um doctest e saber reescrever o teste quando a resposta esperada mudar.

■ Fontes no repositório

lista-de-exercicios-python-brasil/ — 112 exercícios, 6 seções

| Seção | Nº | Exemplos |

|---|---|---|

| secao_01_estrutura_sequencial | 18 | Alo Mundo, soma de 2 números, notas bimestrais, m→cm, área do círculo, Fahrenheit→Celsius, peso ideal, CLT, loja de tintas, tempo de download |

| secao_02_estrutura_de_decisao | 28 | Maior número, positivo/negativo, F ou M, vogal/consoante, 3 números, números decrescentes, turno, salário tabajara, triângulo, equação do 2º grau, ano bissexto, validador de data, decomposição de número, troco, Sherlock, posto de gasolina, hortifruti |

| secao_03_estrutura_de_repeticao | 48 | Número válido, validador de senha, validador de cadastro, crescimento populacional, números 1 a 20, tabuada, Fibonacci, fatorial, número primo, primos menores que N, média aritmética, classificador de turma, urna eletrônica, alunos por turma, coleção de CDs, loja/padaria/conveniência, estatísticas de temperaturas, tabuada com intervalo, sensor de academia, aumento salarial, sala de aula, trânsito, financiamento, lanchonete, eleição presidencial, corretor de provas, salto em distância, inteiro invertido, imprimir série |

| secao_04_exercicios_lista | 13 | Vetor 5 números, vetores inverso, média 4 notas, vetor par e ímpar, calculadora de média, soma de quadrados, intercalação de vetores, filtro de baixinhos, média de temperaturas anual, investigação, melhor jogador, lançamento de dados |

| secao_05_exercicios_funcoes | 2 | Pirâmide, quantidade de dígitos |

| secao_06_exercicios_strings | 5 | Comparador de strings, inversor maiúsculo, nome vertical em escada, validador de CPF (strings) |

SAIBA MAIS: > Achado importante (verificado na auditoria): das 112 questões, 107 são stubs

SAIBA MAIS: > vazios — só 5 têm solução pronta. Isso é uma oportunidade, não um problema:

SAIBA MAIS: > a turma pode resolver e contribuir de verdade.

Projetos reais do autor (para estudar estilo e estrutura)

| Repositório | O que estudar |

|---|---|

| Calculadora/ | Flask + Jinja2, 2 rotas, formulários |

| jogo_da_velha/ | Jogo da velha em console, 5 funções |

| o_acolito/ | Jogo narrativo de aventura, 5 classes |

| Roteiro-de-Estudos/ | App Tkinter de rotina de estudos |

| hubias-estudos/ | SQLite (WASM) no navegador + flashcards + Pomodoro + sync |

| anyms/ | Gestão com IndexedDB, 17 tabelas, custo médio ponderado |

| meu-treino/ | PWA com Wake Lock, GPS, Web Share, localStorage |

| tdm-app/ | Geração e mascaramento de dados de teste com Faker |

| consulta_CNPJ/ | Consumo de 2 APIs, unificando JSON/TXT/Excel |

| o_acolito/ | ■■ o README aponta um nome de script inexistente e PIL/matplotlib não estão declarados — use como aula de documentação × código |

Complementos

- - python-do-zero/14_projetos_praticos/ — 6 projetos prontos
- - python-do-zero/11_tratamento_erros/ — try/except
- - python-do-zero/10_arquivos/ — CSV

EXERCÍCIOS PARA PRATICAR
- laboratorio-tech-social/site/exercicios/desafios-python.html

■ Conceitos-chave

| Conceito | Onde aparece |

|---|---|

| Doctest | Teste escrito dentro do próprio código, como saída esperada de >>> |

EXERCÍCIOS PARA PRATICAR
| Stubs | Função vazia com o enunciado — é o "esqueleto" do exercício |

| try/except | O programa não pode quebrar por causa do usuário |

| Separação entrada/lógica/saída | A mesma estrutura do Visualg, agora em Python |

| Contribuição open source | Fork → branch → commit → Pull Request |

■■ Roteiro (90 min)

Fase 1 — Apresentação (10 min)

EXERCÍCIOS PARA PRATICAR
Gancho: "Esse repositório tem 112 exercícios. 107 estão vazios. Espera. 107

EXERCÍCIOS PARA PRATICAR
exercícios prontos pra você resolver e deixar o mundo melhor — e o primeiro passo é descobrir como funciona um Pull Request."

Quiz de recuperação (B3): "O que input() devolve e por que idade + 1 quebra?"

Fase 2 — Exploração (15 min)

EXERCÍCIOS PARA PRATICAR
Parte 1 — Anatomia de um exercício do repo (7 min).

Abra secao_02_estrutura_de_decisao/ex_17_ano_bissexto.py e mostre a estrutura real:

python

def eh_ano_bissexto(ano: int) -> bool:

"""Retorna True se o ano for bissexto."""

return ano % 4 == 0 and (ano % 100 != 0 or ano % 400 == 0)

def main() -> None:

ano = int(input("Digite um ano: "))

if eh_ano_bissexto(ano):

print(f"{ano} e um ano bissexto")

else:

print(f"{ano} nao e um ano bissexto")

if __name__ == "__main__":

main()

Destaque 4 coisas que a turma ainda não viu:

1. Anotação de tipo (ano: int) -> bool) — o Python anota, mas não obriga.
2. Docstring ("""...""") — a documentação da função.
3. main() — o ponto de entrada do programa.
4. if __name__ == "__main__": — "rode isto só se eu executar o arquivo direto".

Parte 2 — Como o repo valida (8 min).

O repo usa doctest: o teste fica dentro do próprio arquivo.

python

def dobro(n: int) -> int:

"""Retorna o dobro de n.

>>> dobro(2)

"""

return n 2

Rode no terminal:

bash

python -m doctest arquivo.py -v

Explicação: "O doctest é um teste que mora dentro da documentação. Se a documentação mentir, o teste falha. Por isso ele é chamado de documentação executável."

Fase 3 — Mão na massa (35 min)

MÃO NA MASSA
Atividade A — 3 exercícios resolvidos (20 min).

EXERCÍCIOS PARA PRATICAR
Cada aluno escolhe 3 exercícios de seções diferentes:

Seção	Escolha obrigatória (1)	Escolha livre (2)
Sequencial	ex_04_notas_bimestrais ou ex_09_fahrenheit_para_celsius	qualquer um dos 18
Decisão	ex_17_ano_bissexto ou ex_18_validador_de_data	qualquer um dos 28
Repetição	ex_14_qtde_pares_e_impares ou ex_21_numero_primo	qualquer um dos 48
Lista	ex_01_vetor_5_numeros ou ex_05_vetore_par_e_impar	qualquer um dos 13
Funções	ex_01_piramide	—
Strings	ex_09_validador_de_cpf_strings ou ex_02_inversor_maisculo	qualquer um dos 5

EXERCÍCIOS PARA PRATICAR
Processo obrigatório de cada exercício (5 min de papel + 15 min de código):

1. Leia o enunciado e escreva em 1 frase o que o programa deve fazer.
2. Desenhe o fluxograma (o da trilha 8–12, agora com laços e condições).
3. Implemente.

4. Rode o doctest (se houver) e ajuste.
5. Teste com 3 casos: 1 normal, 1 limite, 1 inválido.

MÃO NA MASSA
Atividade B — Blindagem com try/except (15 min).

Todos os programas da turma recebem esta camada:

python

try:

entrada = int(input("Digite um número: "))

except ValueError:

print("Erro: isso não é um número inteiro. Tente de novo.")

entrada = None

E o while True para repetir até o usuário acertar:

python

while True:

try:

n = int(input("Digite um número: "))

break

except ValueError:

print("Não é número. Tente de novo.")

SAIBA MAIS: > Por que isso importa: a turma vai ver na aula 08 (QA) que "o programa não pode

SAIBA MAIS: > quebrar" é o primeiro requisito de qualidade. Faça a ligação: "O try/except que

SAIBA MAIS: > vocês acabaram de escrever é QA dentro do código."

Fase 4 — Projeto (25 min)

EXERCÍCIOS PARA PRATICAR
Desafio: "3 exercícios, 1 programa"

EXERCÍCIOS PARA PRATICAR
Crie um único programa que reúna 3 exercícios do banco em um fluxo com menu:

===== MENU =====

[1] Calculadora de IMC

[2] Verificador de ano bissexto

[3] Tabuada do 7

[4] Sair

Escolha: __

Requisitos:

- - [ ] Menu em while True com break no "Sair"
- - [ ] Cada opção é uma função separada (com docstring)
- - [ ] Validação de entrada em todas as opções (try/except)
- - [ ] Cada função tem pelo menos 1 teste (assert ou doctest)
- - [ ] Um arquivo só, ≤ 150 linhas, com cabeçalho comentado:

Nome do aluno | Data | Descrição

Critério bônus (opcional): transformar o menu em funcionário (você escolhe os

EXERCÍCIOS PARA PRATICAR
exercícios, não o aluno) e escrever o README do programa.

Fase 5 — Mostra (5 min)

2 alunos rodam o programa. Um deles digita letra onde o programa pede número, para provar que a blindagem funciona.

■ Avaliação

| Instrumento | Critério |

|---|---|

EXERCÍCIOS PARA PRATICAR
| 3 exercícios do banco | Resolvidos e testados com 3 casos cada |

| Programa do menu | 4 requisitos do "pronto" |

| Testes | ≥ 1 teste por função |

| Participação | 1 teste ao vivo por aluno |

| Pergunta de saída | "Para que serve o try/except e quando usá-lo?" |

Pergunta de escrita:

> **SAIBA MAIS:** > "Qual a diferença entre corrigir um erro (if/else) e capturar um erro (try/except)?"

Resposta esperada: if/else trata um valor conhecido e previsível (a entrada é

inválida); try/except trata falhas inesperadas (o input não é número, o arquivo não

existe, a rede caiu).

■ Exercícios extras (do repositório)

| Arquivo | Sobre |

|---|---|

| lista-de-exercicios-python-brasil/secao_03_estrutura_de_repeticao/ex_25_classificador_de_turma.py | Classificador de turma (CSV + if/elif) — ótimo pro projeto social |

| .../ex_33_estatisticas_de_temperaturas.py | Estatísticas com listas — base do projeto de dados |

| .../ex_43_lanchonete.py | Lanchonete (caixa, troco) — dá para virar app do projeto final |

| .../ex_44_eleicao_presidencial.py | Eleicao (contagem de votos) - serve para o projeto social |

| .../ex_26_urna_eletronica.py | Urna eletrônica — debate ético perfeito |

| lista-de-exercicios-python-brasil/secao_06_exercicios_strings/ex_09_validador_de_cpf_strings.py | Validador de CPF (algoritmo clássico) |

| python-do-zero/15_api_e_json/ | API + JSON (base do bot de notícias) |

| python-do-zero/12_modulos/ | import, pacotes |

| python-do-zero/13_programacao_orientada_objetos/ | Classes (base do o_acolito) |

| tdm-app/ | Geração de dados de teste com Faker (dados falsos para o projeto final) |

■ Para casa

EXERCÍCIOS PARA PRATICAR
Escolher 3 exercícios novos do banco (de seções que você ainda não fez) e resolver.

EXERCÍCIOS PARA PRATICAR
Comentar os exercícios resolvidos no lista-de-exercicios-python-brasil (se a escola

aceitar) — ou publicar no seu próprio repositório.

■ Próximo módulo

B5 — Web: HTML, CSS e JavaScript

B5 — Web: HTML, CSS e JavaScript (13–17 anos)

Duração: 90 min · Aula 5 de 16

■ Objetivo

Ao final da aula, o aluno será capaz de:

1. Construir uma página semântica (header, nav, main, section, footer) com CSS de layout (Flexbox e Grid) e JavaScript de DOM.
2. Escrever um formulário com validação em HTML e validar em JS.
3. Consumir uma API com fetch e renderizar o resultado.

Objetivo secundário: Explicar a diferença entre HTML, CSS e JS em uma frase cada.

■ Fontes no repositório

EXERCÍCIOS PARA PRATICAR
html-css-js-pratica/ — 15 módulos × 6 exercícios = 90 exercícios, cada um com

"RESULTADO ESPERADO":

EXERCÍCIOS PARA PRATICAR
| Módulo | Conteúdo | Exercícios |

|---|---|---|

| 01_html_basico | Estrutura, tags, atributos, texto, imagens, links | 6 |

| 02_listas_e_tabelas | , <ol>, <li>, <table>, <thead>, <tbody> | 6 |

| 03_css_fundamentos | Seletores, cores, tipografia, box-model, margins, padding | 6 |

| 04_css_layout | Display, position, Flexbox, Grid, responsividade | 6 |

| 05_css_avancado | Pseudo-classes, :hover, :focus, transições, animações, variáveis | 6 |

| 06_html_formularios | <form>, input, label, select, textarea, required, pattern | 6 |

| 07_javascript_basico | Variáveis, tipos, operadores, funções, arrow functions | 6 |

| 08_javascript_dom | getElementById, querySelector, criar/remover elementos | 6 |

| 09_javascript_events | addEventListener, click, submit, keydown, delegação | 6 |

| 10_javascript_assincrono | fetch, Promises, async/await, JSON, tratamento de erro | 6 |

| 11_bootstrap | Grid, componentes, responsividade | 6 |

| 12_projeto_landing_page | Projeto: landing page | 6 |

| 13_projeto_portfolio | Projeto: portfólio | 6 |

| 14_projeto_lista_tarefas | Projeto: lista de tarefas | 6 |

| 15_projeto_jogo_simples | Projeto: jogo simples | 6 |

Projetos reais do autor (mostram o "nível profissional"):

- - portfolio/ — Webpack + API do GitHub + localStorage + CSS de impressão
- - meu-treino/ — PWA com Wake Lock, GPS, Web Share, localStorage
- - anyms/ — IndexedDB, 17 tabelas, custo médio ponderado
- - hubias-estudos/ — SQLite (WASM) no navegador + flashcards + Pomodoro + sync
- - lista_canais/ — agregador de 70+ canais (arrays, filtro, agrupamento)
- - esporte-sao-pedro/ — Supabase + RLS + Edge Functions + IA + SEO
- - barbearia/, Academia/, bem-estar/ — sites estáticos (bons para aula de HTML/CSS puro)
- - instagram-dio-master/ — ■■ clone visual sem nenhum <script>: o README diz "JavaScript" e "botão Entrar", mas o index.html tem 49 linhas e zero JavaScript. Use como aula de "documentação × código".

Trilha web e slides:

laboratorio-tech-social/site/trilhas/adolescentes/02-web.html ·

laboratorio-tech-social/materiais/slides/aula-teens-02-web.html ·

laboratorio-tech-social/site/exercicios/criador-site.html ·

laboratorio-tech-social/site/exercicios/simulador-api.html ·

laboratorio-tech-social/site/exercicios/validador-regex.html

■ Conceitos-chave

| Conceito | Frase que fixa |

|---|---|

| HTML | "A esqueleto. A estrutura. O que existe." |

| CSS | "A aparência. Como cada coisa se apresenta." |

| JavaScript | "O comportamento. O que acontece quando..." |

| DOM | "A árvore de objetos que o navegador fez do seu HTML." |

| Seletor CSS | "Uma regra de "ache quem" para estilizar." |

| Box-model | "Conteúdo + padding + borda + margem = o espaço que o elemento ocupa." |

| Flexbox | "1-dimensional: linha OU coluna." |

| Grid | "2-dimensional: linhas E colunas ao mesmo tempo." |

| async/await | "Pausa o código até a resposta chegar, sem travar a página." |

■■ Roteiro (90 min)

Fase 1 — Apresentação (10 min)

Gancho: "Vocês usam o Instagram 2 horas por dia. Ele é feito de HTML, CSS e JavaScript. E a mesma coisa vale pro site da escola, o do banco e o do governo. A web é a linguagem mais registrada em que existe."

Não diga "não é possível". Diga: "É a mesma base. A diferença é o tamanho e a disciplina."

Demonstração ao vivo (5 min): abra o DevTools (F12) num site qualquer e mostre:

- - Aba Elements → o HTML (a árvore)
- - Aba Styles → o CSS aplicado

- - Aba Console → o JavaScript

Diga: "Três abas, três linguagens. Todos os sites do mundo são isso."

Fase 2 — Exploração (20 min)

Parte 1 — Anatomia de uma página (8 min).

html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Minha Página</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header><h1>Título</h1></header>
<nav><a href="#">Link</a></nav>
<main>
<section>
<h2>Seção</h2>
<p>Texto</p>
<form id="form">
<label for="nome">Nome:</label>
<input id="nome" type="text" required>
<button type="submit">Enviar</button>
</form>
</section>
</main>
```

<footer> © 2026 </footer>

<script src="script.js"></script>

</body>

</html>

Explique por que viewport está no <head>: sem isso, o site aparece minúsculo no celular.

Parte 2 — Box-model e Flexbox (7 min).

Desenhe o box-model no quadro:

margem (margin)

■■■■■■■■■■■■■■■■■■■■■■■■

■ borda (border) ■

■ ■■■■■■■■■■■■■■■■■■ ■

■ ■ padding ■ ■

■ ■ ■■■■■■■■■■■■ ■ ■

■ ■ ■ conteúdo ■ ■ ■

■ ■ ■■■■■■■■■■■■ ■ ■

■ ■■■■■■■■■■■■■■■■■■ ■

■■■■■■■■■■■■■■■■■■■■■■■■

Flexbox em 1 linha:

.container { display: flex; gap: 1rem; }

.card { flex: 1; padding: 1rem; border-radius: 8px; }

Grid em 1 linha:

.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }

Parte 3 — DOM e eventos (5 min).

javascript

const botao = document.querySelector('#botao');

const saida = document.querySelector('#saida');

botao.addEventListener('click', () => {

saida.textContent = 'Cliquei!';

Parte 4 — fetch (0 min, só mostrar)

javascript

async function carregar() {

const r = await fetch('https://api.exemplo.com/dados');

if (!r.ok) throw new Error('HTTP ' + r.status);

const dados = await r.json();

console.log(dados);

Fase 3 — Mão na massa (40 min)

MÃO NA MASSA
Atividade A — Landing page (25 min).

Base: html-css-js-pratica/12_projeto_landing_page/.

Construa uma página de 1 seção com:

- - header com título e subtítulo
- - nav com 3 links
- - 3 cards em Flexbox (depois em Grid)
- - 1 form de contato com required e type="email"
- - 1 botão que mostra uma mensagem sem recarregar a página (addEventListener + preventDefault)

MÃO NA MASSA
Atividade B — 3 exercícios do repo (15 min).

Escolha 3 entre:

- - 03_css_fundamentos/exercicio_03_box_model — box-model
- - 04_css_layout/exercicio_02_flexbox — Flexbox
- - 04_css_layout/exercicio_03_grid — Grid
- - 06_html_formularios/exercicio_04_validacao — validação
- - 08_javascript_dom/exercicio_03_criar_elementos — criar elementos
- - 09_javascript_events/exercicio_02_submit — evento de submit

- - 10_javascript_assincrono/exercicio_02_fetch — fetch

Cada um tem um "RESULTADO ESPERADO" no repo. Cole-o como comentário no código e confirme que a página faz o que dice.

MÃO NA MASSA
Atividade C — Consome uma API (extra, 0 min em aula / casa).

Use laboratorio-tech-social/site/exercicios/simulador-api.html para entender o formato, e depois a API real do GitHub (usada pelo repo portfolio/):

javascript

```
const resposta = await fetch('https://api.github.com/users/Marquezbertin/repos?per_page=5');
const repos = await resposta.json();
```

Fase 4 — Projeto (15 min)

EXERCÍCIOS PARA PRATICAR
Desafio: "A página do projeto final"

Monte a landing page do projeto que a turma vai entregar (aula 13):

- - Nome do projeto, slogan, 3 cards com os "pilares"
- - 1 formulário de "quero ajudar"
- - 1 botão que mostra a data de atualização (via new Date())

Critério de pronto:

- - [ ] HTML semântico (usa header, nav, main, section, footer)
- - [ ] viewport no <head>
- - [ ] Layout com Flexbox ou Grid
- - [ ] 1 interação com JS (formulário, botão, menu)
- - [ ] Passa no validador: sem erro no console (F12 → Console → 0 erros)

Fase 5 — Mostra (5 min)

Abra 2 landing pages no projetor. Cada turma dá 1 ponto forte e 1 sugestão.

■ Avaliação

| Instrumento | Critério |

|---|---|

| Landing page | 5 critérios do "pronto" |

EXERCÍCIOS PARA PRATICAR
| 3 exercícios do repo | "RESULTADO ESPERADO" conferido |

| Console limpo | 0 erros no DevTools |

| Semanticidade | Aluno sabe dizer o que é main e por que existe |

| Pergunta de saída | "O que o preventDefault() faz?" |

Pergunta de escrita:

SAIBA MAIS: > "Você tem um <form> que envia para o servidor. Ao clicar em enviar, a página recarrega

SAIBA MAIS: > e o texto digitado se perde. Como resolver sem recarregar?"

SAIBA MAIS: > Resposta: event.preventDefault() no handler do submit + AJAX (fetch) ou

SAIBA MAIS: > manipulação local.

■ Exercícios extras (do repositório)

| Arquivo | Sobre |

|---|---|

| html-css-js-pratica/05_css_avancado/ | Pseudo-classes, transições, variáveis CSS |

| html-css-js-pratica/11_bootstrap/ | Bootstrap (grid e componentes) |

| html-css-js-pratica/13_projeto_portfolio/ | Portfólio — o projeto mais próximo do "portfólio do aluno" |

| html-css-js-pratica/15_projeto_jogo_simples/ | Jogo simples (canvas ou DOM) |

| portfolio/ (repo do autor) | Webpack, API do GitHub, CSS de impressão |

| lista_canais/ | Agregador: arrays, filtro, agrupamento, ordenação |

| meu-treino/ | PWA: manifest, service worker, Wake Lock, GPS, Web Share |

| anyms/ | IndexedDB (banco no navegador) + 17 tabelas |

| laboratorio-tech-social/site/exercicios/criador-site.html | Gerador de site (para quem travou) |

| laboratorio-tech-social/site/exercicios/validador-regex.html | Regex no navegador |

| laboratorio-tech-social/site/exercicios/gerador-dados.html | Gerar dados fictícios para popular o site |

■ Para casa

Transformar a landing page em site de 3 páginas (index, sobre, contato) com CSS

externo em arquivo próprio. Publicar no GitHub Pages depois da aula 07.

■ Próximo módulo

B6 — Banco de Dados e SQL

GLOSSÁRIO

TERMO	SIGNIFICADO
Algoritmo	Sequência de passos ordenados para resolver um problema
Programa	Conjunto de instruções que o computador executa
Bug	Erro ou falha em um programa
Hardware	Componentes físicos do computador
Software	Programas e sistemas utilizados no computador
Internet	Rede mundial de computadores interligados
Navegador	Programa para acessar e visualizar páginas da web
Variável	Espaço para armazenar dados na memória
Função	Bloco de código reutilizável que executa uma tarefa
Loop (Laço)	Repetição de instruções enquanto uma condição for verdadeira
Lógica	Raciocínio para tomar decisões e resolver problemas
Git	Sistema para controlar versões de arquivos e projetos
GitHub	Plataforma online para hospedar projetos com Git
Banco de Dados	Conjunto organizado de informações
SQL	Linguagem para gerenciar bancos de dados
HTML	Linguagem para estruturar páginas web
CSS	Linguagem para estilizar páginas web
JavaScript	Linguagem para dar interatividade à web
QA (Quality Assurance)	Garantia da qualidade de software através de testes